Boyi Wei, Kaixuan Huang, Yangsibo Huang, Tinghao Xie, Xiangyu Qi, Mengzhou Xia, Prateek Mittal, Mengdi Wang, and Peter Henderson. Assessing the brittleness of safety alignment via pruning and low-rank modifications. *International Conference on Machine Learning (ICML)*, 2024.

Jason Wei and Kai Zou. EDA: Easy data augmentation techniques for boosting performance on text classification tasks. In Kentaro Inui, Jing Jiang, Vincent Ng, and Xiaojun Wan (eds.), *Proceedings of the 2019 Conference on Empirical Methods in Natural Language Processing and the 9th International Joint Conference on Natural Language Processing (EMNLP-IJCNLP)*, pp. 6382–6388, Hong Kong, China, November 2019. Association for Computational Linguistics. doi: 10.18653/v1/D19-1670. URL https://aclanthology.org/D19-1670.

Zeming Wei, Yifei Wang, and Yisen Wang. Jailbreak and guard aligned language models with only few in-context demonstrations. *arXiv preprint arXiv:2310.06387*, 2023b.

Thomas Wolf, Lysandre Debut, Victor Sanh, Julien Chaumond, Clement Delangue, Anthony Moi, Pierric Cistac, Tim Rault, Remi Louf, Morgan Funtowicz, Joe Davison, Sam Shleifer, Patrick von Platen, Clara Ma, Yacine Jernite, Julien Plu, Canwen Xu, Teven Le Scao, Sylvain Gugger, Mariama Drame, Quentin Lhoest, and Alexander Rush. Transformers: State-of-the-art natural language processing. In Qun Liu and David Schlangen (eds.), *Proceedings of the 2020 Conference on Empirical Methods in Natural Language Processing: System Demonstrations*, pp. 38–45, Online, October 2020. Association for Computational Linguistics. doi: 10.18653/v1/2020.emnlp-demos.6. URL https://aclanthology.org/2020.emnlp-demos.6.

An Yang, Baosong Yang, Binyuan Hui, Bo Zheng, Bowen Yu, Chang Zhou, Chengpeng Li, Chengyuan Li, Dayiheng Liu, Fei Huang, Guanting Dong, Haoran Wei, Huan Lin, Jialong Tang, Jialin Wang, Jian Yang, Jianhong Tu, Jianwei Zhang, Jianxin Ma, Jianxin Yang, Jin Xu, Jingren Zhou, Jinze Bai, Jinzheng He, Junyang Lin, Kai Dang, Keming Lu, Keqin Chen, Kexin Yang, Mei Li, Mingfeng Xue, Na Ni, Pei Zhang, Peng Wang, Ru Peng, Rui Men, Ruize Gao, Runji Lin, Shijie Wang, Shuai Bai, Sinan Tan, Tianhang Zhu, Tianhao Li, Tianyu Liu, Wenbin Ge, Xiaodong Deng, Xiaohuan Zhou, Xingzhang Ren, Xinyu Zhang, Xipin Wei, Xuancheng Ren, Xuejing Liu, Yang Fan, Yang Yao, Yichang Zhang, Yu Wan, Yunfei Chu, Yuqiong Liu, Zeyu Cui, Zhenru Zhang, Zhifang Guo, and Zhihao Fan. Qwen2 technical report. *arXiv preprint: arXiv:2407.10671*, 2024.

Youliang Yuan, Wenxiang Jiao, Wenxuan Wang, Jen tse Huang, Pinjia He, Shuming Shi, and Zhaopeng Tu. GPT-4 is too smart to be safe: Stealthy chat with LLMs via cipher. *International Conference on Learning Representations (ICLR)*, 2024.

Andy Zou, Zifan Wang, J Zico Kolter, and Matt Fredrikson. Universal and transferable adversarial attacks on aligned language models. *arXiv preprint arXiv:2307.15043*, 2023.

APPENDIX

A LIMITATIONS

While the small model trained with our HarmAug method significantly improves efficiency over larger models and yields comparable performance, there are still some limitations to our approach. First, the performance gains diminish as the size of the synthetic dataset increases. This may be attributed to the independent sampling of harmful instructions by the LLM. The lack of awareness of previously generated samples may result in redundant instances after multiple iterations. Steering the LLM to consistently generate new examples would be an interesting direction for future work. Another limitation is that FlashAttention (Dao et al., 2022) cannot be applied to DeBERTa due to its use of disentangled attention, which differs from the standard attention mechanism optimized by FlashAttention. Optimizing DeBERTa's attention could further reduce latency.

B IMPLEMENTATION DETAILS

B.1 MODEL SELECTION

We chose DeBERTa-v3-large based on the following three criteria. First, we prefer a bidirectional encoder over an autoregressive decoder-only model, as predicting the harmfulness of a prompt is a binary classification task rather than complex sequence generation. Second, among bidirectional encoders, we select the model based on its overall performance on general benchmark datasets, such as GLUE (Wang et al., 2024) and SQuAD (Rajpurkar et al., 2016). Finally, we choose the largest sub-billion-parameter model within the model family.

B.2 GFLOWNET BASELINE

Following Lee et al. (2024), we fine-tune GPT-2 with 124 million parameters on prompts from the AdvBench dataset (Zou et al., 2023) with maximum likelihood estimation. We use the AdamW (Loshchilov & Hutter, 2019) optimizer with a learning rate $3 \cdot 10^{-5}$, a batch size 1024 and linear decay of learning rate. Then we fine-tune GPT-2 with trajectory balance objective (Malkin et al., 2022), using the reward function defined as:

$$R(\mathbf{x}) = p_\theta(c = 1 \mid \mathbf{x})^{1/\beta} \cdot p_{\texttt{ref}}(\mathbf{x})^{1/\gamma},$$

where $p_\theta(c = 1 \mid \mathbf{x})$ is the probability of the prompt $\mathbf{x}$ being harmful using Llama-Guard-3 and $p_{\texttt{ref}}$ is the initial fined-tuned GPT-2 model to measure the naturalness of the prompt. During GFlowNet training, prompts are sampled using either on-policy or off-policy strategies with a probability of 0.5. For the on-policy strategy, we uniformly select a temperature from $[0.5, 2.0]$ and sample prompts using GPT-2 with the chosen temperature. For the off-policy strategy, we sample prompts from a replay buffer. We use the AdamW optimizer for GFlowNet fine-tuning with $50{,}000$ steps, a batch size of 64, $\beta = 0.1$, and $\gamma = 1.0$. After that, we sample $100{,}000$ prompts for augmenting the training dataset $\mathcal{D}$.

B.3 RED-TEAMING

We use the same training objective and hyperparameters as for the GFlowNet fine-tuning described in Appendix B.2, with the exception that the reward function defined in Eq. (3) is used and the teacher model p_θ is replaced by the student q_ϕ. To approximate the true log reward, we sample 5 responses from the target LLM as follows:

$$\log R(\mathbf{x}) \approx \frac{1}{5\beta} \sum_{i=1}^{5} \log q_\phi(c = 1 \mid \mathbf{x}, \mathbf{y}^{(i)}) + \frac{1}{\gamma} \log p_{\texttt{ref}}(\mathbf{x}), \quad \mathbf{y}^{(i)} \overset{\text{iid}}{\sim} p_{\texttt{target}}(\mathbf{y} \mid \mathbf{x}).$$

However, GFN suffers from mode collapse due to the safety alignment of the target LLM. The safety-tuned target LLM refuses to generate responses to most attack prompts, leading to sparse rewards. To tackle this challenge, following Lee et al. (2024), we collect high-reward prompts sampled

during GFlowNet fine-tuning and re-train the initially fine-tuned GPT-2 model to maximize the log-likelihood of the collected samples for 1,000 steps. In this stage, we use the AdamW (Loshchilov & Hutter, 2019) optimizer with a batch size of 1,024, and a learning rate of 10^{-4}. The learning rate is linearly decayed from the initial value to 0.

C Evaluation Metric

F1 score. The F1 score is a measure of a model's accuracy, balancing precision and recall. It is defined as the harmonic mean of precision and recall. Precision is the ratio of correctly predicted positive instances (true positives) to all predicted positive instances (union of true positives and false positives):

$$\text{Precision} = \frac{\text{TP}}{\text{TP} + \text{FP}},$$

where TP and FP denote the number of true positives and false positives, respectively.

Recall is the ratio of correctly predicted positive instances (true positives) to all actual positive instances (union of true positives and false negatives):

$$\text{Recall} = \frac{\text{TP}}{\text{TP} + \text{FN}},$$

where FN denotes the number of false negatives. The formula for the F1 score is defined as:

$$\text{F1} = 2 \times \frac{\text{Precision} \times \text{Recall}}{\text{Precision} + \text{Recall}}$$

Area Under Precision-Recall Curve (AUPRC). The Area Under the Precision-Recall Curve (AUPRC) is defined as the integral of the precision with respect to recall:

$$\text{AUPRC} = \int_0^1 P(R)dR,$$

where the term $P(R)$ refers to precision as a function of recall. This means that for each value of recall $R \in [0,1]$ with the decision threshold of the classifier, $P(R)$ gives the corresponding precision value. In practice, since precision and recall are not continuous functions but rather vary discretely based on the decision threshold of the classifier, $P(R)$ typically represents the precision at each level of recall for various thresholds. The precision-recall curve is plotted with recall on the x-axis and precision on the y-axis. A higher AUPRC value indicates that the model performs better at distinguishing between positive and negative classes across various thresholds.

D Additional Results

D.1 Ablation Study of Empty Response

To study the effect of including the empty sequences $\hat{\mathbf{y}}_{j3}$, we remove all of them in the synthetic dataset $\hat{\mathcal{D}}$ and train DeBERTa-v3-large. As shown in Table 8, removing the empty sequences significantly degrades the performance on most of the benchmark datasets except for F1 score on OAI. This results shows the importance of including the empty responses to train the model to handle both instruction and instruction-response pair classification tasks.

Table 8: Ablation of empty responses $\hat{\mathbf{y}}_{j3}$ in our synthetic dataset.

	OAI		**ToxicChat**		**HarmBench**		**WildGuardMix**		**Average**	
Model	F1	AUPRC	F1	AUPRC	F1	AUPRC	F1	AUPRC	F1	AURPC
w/o empty response	**0.7629**$_{\pm 0.0130}$	0.8477$_{\pm 0.0085}$	0.4935$_{\pm 0.0128}$	0.5132$_{\pm 0.0095}$	0.8341$_{\pm 0.0042}$	0.8705$_{\pm 0.0041}$	0.7494$_{\pm 0.0147}$	0.8210$_{\pm 0.0040}$	0.7100$_{\pm 0.0080}$	0.7631$_{\pm 0.0009}$
w/ empty response	0.7236$_{\pm 0.0084}$	**0.8791**$_{\pm 0.0032}$	**0.6283**$_{\pm 0.0144}$	**0.7553**$_{\pm 0.0101}$	0.8331$_{\pm 0.0009}$	**0.8841**$_{\pm 0.0035}$	**0.7576**$_{\pm 0.0144}$	**0.8265**$_{\pm 0.0135}$	**0.7357**$_{\pm 0.0076}$	**0.8362**$_{\pm 0.0056}$